Project 2: A Simple Web Server in Python

Course: EE129 Computer Networks

Instructor: Prof. Almir Davis

Institution: Tufts University, Department of Electrical and Computer Engineering

Individual Assignment: Unlike Project 1, which was completed in groups of two (or three), Project 2 is an individual assignment. Every student must independently write, test, and submit their own original code and report.

1 Introduction

Python is a popular, general-purpose programming language that is easy to learn and program. Web technology is a key technology that revolutionized the Internet in the 1990s and has been fueling the development of the Internet for more than 20 years. A web server and a client (web browser) communicate with a TCP-based protocol called HTTP (Hypertext Transfer Protocol). We will use Python to write a simple web server from scratch!

2 Getting Started

1. Download Python Here if you don't have Python3 installed on your computer. Ideally you should have the latest version of Python installed, but any Python3 version is fine.
2. Optionally, install an environment manager like Anaconda.
3. Review helpful resources on socket programming and HTML • Study the Python documentation or tutorials for socket programming in Python. Socket is the API (Application Programming Interface) for various network protocols, including the TCP/IP protocol suite
   - Study the HTTP protocol by reading Section 7.3 of your textbook.
   - Study the HTML tutorials to learn the basics of writing a web page. Pay attention to HTML forms. We will use an HTML form to send data from the browser to the web server.

3 Running a basic server

Download and run the starter code (server.py). Link to the starter code on GitHub Questions to Consider (not required to answer):

- What does the program do?
- How does a web client make a request to the server?

- What happens when the client specifies an incorrect path?
- Which lines are causing the program to block/wait?

Modify the starter code to send an image file to the client when requested.

4 Build your own web server

1. Write a simple web server using sockets in Python that can communicate with a web browser in HTTP 1.1 protocol. When specifying the TCP port, use a port number in the range of 1024-65535 so that no special privilege is needed for your server. You are not allowed to use any libraries that go against the spirit of the assignment or would make this project trivial (e.g. Flask). You should primarily use the socket library for communication. Add the following line at the beginning of your code to use the socket library.

   ```
   import socket
   ```

2. When requested by a web browser, your web server should check the URL sent by your browser and act accordingly.

   (a) If the URL is just "/", it should display a page with a form for the user to select from 2 different colors (red, green), then submit it to your web server with the GET method

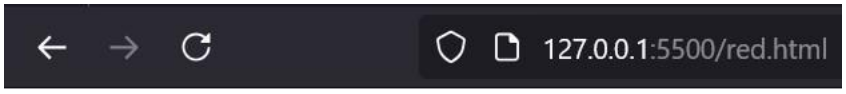
   

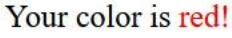
   *Path in the image should be 127.0.0.1:5500/

   (b) If the URL has the keyword "red", it should send back a web page with a sentence

   Your color is red!

   The word red must be colored red.

   (c) If the URL has the keyword "green", it should send back a web page with a sentence

   Your color is green!

The word green must be colored green.

(d) If the URL is ”/login”, show a login form (see login system directions)

(e) Otherwise, the user requested a page that does not exist. Return the status code 404 to the web browser.

5 A simple login system

Build a simple login page with a form that contains fields for a username and password. When the form is submitted send a POST request to the server with the entered username and password. On the server:

- Check if the entered credentials are valid. It’s up to you to decide what are considered valid credentials.
- If the credentials are invalid, send an error to the client.
- If the credentials are valid, redirect the client to a HTML page that contains the number of successful and unsuccessful login attempts. The number of successful/unsuccessful attempts should be dynamic values and should remain accurate, even as the server is restarted. (Hint: You can store these values in a file)

6 Enhancing your web server

You might notice that in the starter code that the mechanism for closing the server isn’t very clean. In fact, Ctrl-C oftentimes will not close the server until the client makes a new request. Let’s add a command line tool to close the server more effectively.

If the server detects an input over stdin (user input), it should respond using the following logic:

- If the input is equal to exit, close all open sockets and exit the program.
- If the input is equal to help, print a guide on the commands your server supports to stdout. At the minimum, print information on how to exit the server.
- If the input does not match a supported command on your server print out {CMD}: Command Not Found where {CMD} is the command name.

The server must respond to command line input immediately. The server should process user input, socket activity, and command line output in parallel. The select library (and syscall) might be helpful in achieving this behavior.

7 Tips and Extra Credit

- The total length of your program should be less than 200 lines (excluding comments). If your program is too long, it is overly complicated.

- Run your program by typing python filename or python3 filename depending on your installation method. Or for some platforms, you can just double click your source file to run it.

- If your program fails because the specified port has been used by other programs, change your port number in your code and run it again. Alternatively, you can set a flag to enable the port be reused:

```
# Replace `serversocket` with your own socket instance
serversocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

- You can visit your web server from a browser on the same computer by using the special domain name localhost. Or you can visit your web server from another computer on the same LAN by using your computer's IP address.

- Use your imagination to add more features to the project to get extra credit. Describe them in your report. If you are unsure of what counts for extra credit, please ask. Please use a separate section of the report titled "Extra Credit" to describe your additional features.

8 Submission

Submit your code and a report including an introduction, how it works, the results and discussion with a well-commented program. Please submit your written report as a single Word or PDF file and all other files including python code files as a single zip file.

Enjoy your project. Have fun!